



Tratamento de Exceções em C++



Tratamento de Exceções

- É mecanismo existente em algumas linguagens de programação para tratar erros e exceções em tempo de execução
- Sem o tratamento de exceções, qualquer erro simples **poderia interromper a execução** do programa
- A ideia é padronizar e ter uma forma adequada para lidar com erros
- Permitir que o programa:
 - Detecte quando algo deu errado;
 - Intercepte o erro antes que ele cause uma falha grave;
 - Tome uma ação apropriada, como:
 - Mostrar uma mensagem clara ao usuário;
 - Recuperar-se do erro e continuar a execução;
 - Registrar o **erro (log)** para análise futura;
 - Encerrar de forma controlada, se necessário.

Como funciona ?

A maioria das linguagens usa três conceitos básicos

try - Bloco de código que “pode” lançar um erro

throw - Instrução usada para lançar uma exceção

catch - Bloco de código que captura e trata a exceção que foi lançada

Exemplo

Um dos grandes objetivos é separar o código da lógica de negócio do código que seria só responsável por tratar erros

```
try {  
    // código que pode dar erro  
    resultado = dividir(a, b);  
}  
catch (DivisaoPorZero) {  
    // tratamento para divisão por zero  
    mostrar("Não é possível dividir por zero.");  
}
```

Significa → "o código envolvido pelo bloco *try* pode lançar exceções. Se lançar elas serão capturadas e tratadas dentro do bloco *catch*".
A ideia → não ficar tratando exceções no mesmo bloco de código da lógica de negócio.

```
#include <iostream>
using namespace std;
```

Exemplo em C++

```
// Função que lança exceção se o divisor for zero
double dividir(double numerador, double denominador) {
    if (denominador == 0) {
        throw "Erro: divisão por zero!";
    }
    return numerador / denominador;
}
```

```
int main() {
    double a, b;

    cout << "Digite o numerador: ";
    cin >> a;
    cout << "Digite o denominador: ";
    cin >> b;
```

```
    try {
        double resultado = dividir(a, b);
        cout << "Resultado: " << resultado << endl;
    }
```

```
    catch (const char* mensagemDeErro) {
        cerr << mensagemDeErro << endl;
    }
```



```
    cout << "Programa não quebra ... continua a execução normalmente e finaliza ...". << endl;
    return 0;
}
```

- A função dividir() lança uma exceção dentro dela
- Todo lugar que usar a função dividir() deverá tratar a possível exceção lançada com try/catch
- Se não tratar, pode gerar um erro inesperado de qq forma.
- Note que o programa continua sua execução normalmente pq o erro foi tratado, isto é, o erro não interrompe a execução do programa

Observe que a estrutura é sempre a seguinte:

- 1 - Há uma função que lança uma exceção (throw).
- 2 - O código cliente, que usa a função, precisa tratar a exceção que a função pode lançar

```
#include <iostream>
using namespace std;
```

Exemplo em C++

```
// Função que lança exceção se o divisor for zero
double dividir(double numerador, double denominador) {
    if (denominador == 0) {
        throw "Erro: divisão por zero!";
    }
    return numerador / denominador;
}

int main() {
    double a, b;

    cout << "Digite o numerador: ";
    cin >> a;
    cout << "Digite o denominador: ";
    cin >> b;

    try {
        double resultado = dividir(a, b);
        cout << "Resultado: " << resultado << endl;
    }
    catch (const char* mensagemDeErro) {
        cerr << mensagemDeErro << endl;
    }

    cout << "Programa continua a execução normalmente e finaliza ..". << endl;
    return 0;
}
```

- o "**cerr**" também é usado para imprimir mensagens, assim como o cout
- mas o cerr é uma boa prática para mostrar que é uma mensagem de erro
- o cerr exibe as mensagens instantaneamente no console, enquanto o cout primeiro envia elas para um buffer para depois exibir

```

#include <iostream>
#include <stdexcept> // para std::runtime_error
using namespace std;

class ContaBancaria {
private:
    string titular;
    double saldo;

public:
    ContaBancaria(string nome, double saldoInicial) : titular(nome), saldo(saldoInicial) { }

    void depositar(double valor) {
        if (valor <= 0) {
            throw invalid_argument("Valor de depósito deve ser positivo.");
        }
        saldo += valor;
    }

    void sacar(double valor) {
        if (valor <= 0) {
            throw invalid_argument("Valor de saque deve ser positivo.");
        }
        if (valor > saldo) {
            throw runtime_error("Saldo insuficiente para saque.");
        }
        saldo -= valor;
    }

    void exibirSaldo() const {
        cout << "Saldo atual de " << titular << ": R$ " << saldo << endl;
    }
};

```

classes da
biblioteca
stdexcept
(padrão do C++)

Exemplo com classe OO

//dentro do mesmo arquivo

```

int main() {
    ContaBancaria conta("João", 500.0);

    try {
        conta.exibirSaldo();

        conta.depositar(200); //testar com -200
        conta.exibirSaldo();

        conta.sacar(1000); // Vai lançar exceção //testar com valor negativo

        conta.exibirSaldo(); // Não será executado se houver erro

    } catch (const invalid_argument& e) {
        cerr << "Erro de argumento: " << e.what() << endl;

    } catch (const runtime_error& e) {
        cerr << "Erro em tempo de execução: " << e.what() << endl;
    }

    cout << "Programa encerrado com segurança." << endl;

    return 0;
}

```

```

#include <iostream>
#include <stdexcept> // para std::runtime_error
using namespace std;

class ContaBancaria {
private:
    string titular;
    double saldo;

public:
    ContaBancaria(string nome, double saldoInicial) : titular(nome), saldo(saldoInicial) { }

    void depositar(double valor) {
        if (valor <= 0) {
            throw invalid_argument("Valor de depósito deve ser positivo.");
        }
        saldo += valor;
    }

    void sacar(double valor) {
        if (valor <= 0) {
            throw invalid_argument("Valor de saque deve ser positivo.");
        }
        if (valor > saldo) {
            throw runtime_error("Saldo insuficiente para saque.");
        }
        saldo -= valor;
    }

    void exibirSaldo() const {
        cout << "Saldo atual de " << titular << ": R$ " << saldo << endl;
    }
};

```

criando objeto do tipo da classe - chamando construtor e passando uma string.

Exemplo com classe OO

//dentro do mesmo arquivo

```

int main() {
    ContaBancaria conta("João", 500.0);

    try {
        conta.exibirSaldo();

        conta.depositar(200); //testar com -200
        conta.exibirSaldo();

        conta.sacar(1000); // Vai lançar exceção //testar com valor negativo

        conta.exibirSaldo(); // Não será executado se houver erro
    } catch (const invalid_argument& e) {
        cerr << "Erro de argumento: " << e.what() << endl;
    }

    } catch (const runtime_error& e) {
        cerr << "Erro em tempo de execução: " << e.what() << endl;
    }

    cout << "Programa encerrado com segurança." << endl;

    return 0;
}

```

objeto que foi criado e que é recebido aqui no tratamento

```

#include <iostream>
#include <stdexcept> // para std::runtime_error
using namespace std;

class ContaBancaria {
private:
    string titular;
    double saldo;

public:
    ContaBancaria(string nome, double saldoInicial) : titular(nome), saldo(saldoInicial) { }

    void depositar(double valor) {
        if (valor <= 0) {
            throw invalid_argument("Valor de depósito deve ser positivo.");
        }
        saldo += valor;
    }

    void sacar(double valor) {
        if (valor <= 0) {
            throw invalid_argument("Valor de saque deve ser positivo.");
        }
        if (valor > saldo) {
            throw runtime_error("Saldo insuficiente para saque.");
        }
        saldo -= valor;
    }

    void exibirSaldo() const {
        cout << "Saldo atual de " << titular << ": R$ " << saldo << endl;
    }
};

```

criando objeto do tipo da classe - chamando construtor e passando uma string.

Exemplo com classe OO

O método what é um método virtual da classe std::exception

virtual const char* what() const noexcept;

```

//...
int main() {
    ContaBancaria conta("João da Silva", 1000);

    conta.exibirSaldo();

    conta.depositar(200); //testar com +200
    conta.exibirSaldo();

    conta.sacar(1000); // Vai lançar exceção //testar com valor negativo

    conta.exibirSaldo(); // Não será executado se houver erro

    try {
        conta.depositar(-200);
    } catch (const invalid_argument& e) {
        cerr << "Erro de argumento: " << e.what() << endl;
    }

    try {
        conta.sacar(1500);
    } catch (const runtime_error& e) {
        cerr << "Erro em tempo de execução: " << e.what() << endl;
    }

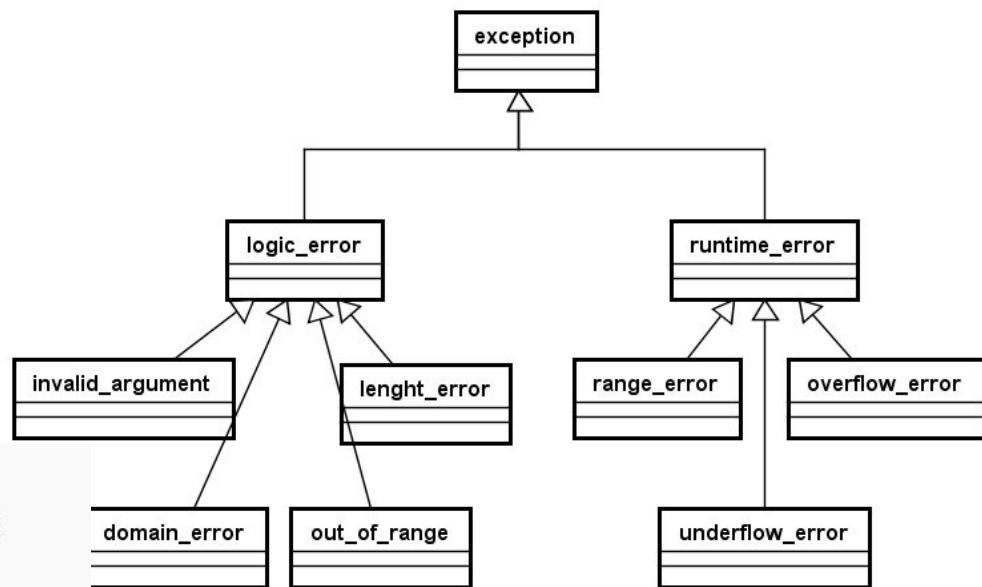
    cout << "Programa encerrado com segurança." << endl;

    return 0;
}

```

As classes filhas não possuem métodos específicos

As classes mais especializadas servem como "rótulos" para tipos de erros



`std::exception`

- └─ `std::logic_error` ← erro de lógica detectável
 - └─ `std::invalid_argument` ← argumento inválido
 - └─ `std::domain_error` ← erro de domínio matemático
 - └─ `std::length_error` ← tamanho inválido de estrutura
 - └─ `std::out_of_range` ← acesso fora do intervalo
- └─ `std::runtime_error` ← erro em tempo de execução
 - └─ `std::range_error` ← erro numérico de intervalo
 - └─ `std::overflow_error` ← estouro numérico
 - └─ `std::underflow_error` ← subfluxo numérico

o C++ lança exceções sozinho ?

```
#include <vector>
#include <iostream>
using namespace std;

int main() {
    vector<int> v = {1, 2, 3};

    try {
        cout << v.at(5) << endl; // índice inválido!

    } catch (const out_of_range& e) {
        cerr << "Exceção: " << e.what() << endl;
    }
}
```

- Ao acessar o índice inválido com **at()**, o C++ lança uma exceção do tipo **out_of_range**
- Significa que dentro do método **at()** há um **throw** que lança essa exceção
- Acessar índices de array com **[]** não lança exceção

o C++ lança exceções sozinho ?

- Stoi() → **String to Integer**
- Conversão de String para número também lança exceção
- A exceção lançada é do tipo `invalid_argument`

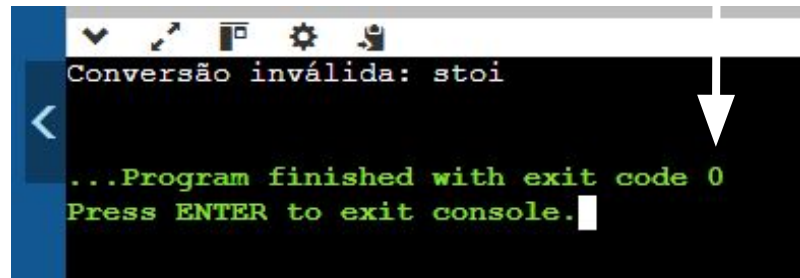
```
#include <string>
#include <iostream>
using namespace std;
```

```
int main() {
    string s = "abc";
    try {
```

```
        int x = stoi(s); // tentativa de conversão inválida
```

```
    } catch (const invalid_argument& e) {
        cerr << "Conversão inválida: " << e.what() << endl;
    }
```

```
}
```



```
Conversão inválida: stoi
...Program finished with exit code 0
Press ENTER to exit console.
```

o C++ lança exceções sozinho ?

```
#include <string>
#include <iostream>
using namespace std;

int main() {
    string s = "abc";
    try {

        int x = stoi(s); // tentativa de conversão inválida

    } catch (const invalid_argument& e) {
        cerr << "Conversão inválida: " << e.what() << endl;
    }
}
```

Sem o tratamento da exceção, a execução resultaria em:

```
terminate called after throwing an instance
of 'std::invalid_argument'
```

```
what():  stoi
```

```
...Program finished with exit code 134
Press ENTER to exit console.
```

o C++ lança exceções sozinho ?

- Exceção do tipo **bad_alloc** é lançada ao tentar criar um array gigantesco

```
#include <iostream>
using namespace std;

int main() {
    try {
        int* p = new int[10000000000000]; // pode falhar!

    } catch (const bad_alloc& e) {

        cerr << "Erro de alocação: " << e.what() << endl;

    }
}
```

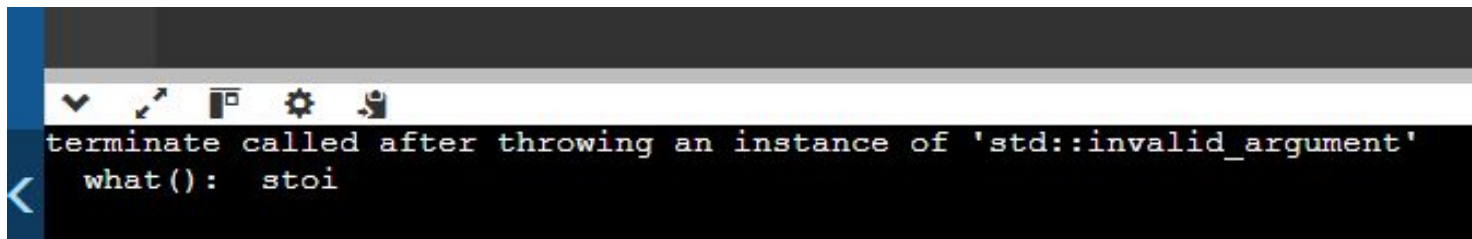
Situações que lançam exceções

Situação	Exceção lançada	Exemplo em C++
Acesso fora do intervalo com <code>std::vector::at()</code> ou <code>std::string::at()</code>	<code>std::out_of_range</code>	<code>v.at(10); // se v.size() < 11</code>
Acesso a chave inexistente com <code>std::map::at()</code> ou <code>std::unordered_map::at()</code>	<code>std::out_of_range</code>	<code>m.at("chave"); // se 'chave' não existir</code>
Conversão inválida de string para número com <code>std::stoi</code> , <code>std::stof</code> , etc.	<code>std::invalid_argument</code>	<code>stoi("abc"); // string não numérica</code>
Valor numérico fora do intervalo durante conversão com <code>std::stoi</code> , etc.	<code>std::out_of_range</code>	<code>stoi("99999999999999999999"); // valor muito grande</code>
Criação de <code>bitset</code> com string inválida	<code>std::invalid_argument</code>	<code>bitset<8>("10201010"); // caractere inválido '2'</code>
Violação de pré-condições internas em classes da STL	<code>std::logic_error</code> (ou derivadas)	<code>std::vector<int> v; auto it = v.begin(); cout << *it; // acesso inválido a iterador</code>

Como saber qual exceção uma função lança ?

Duas formas de saber:

1 - Durante a execução, quando um erro acontecer. Exemplo:

A screenshot of a C++ IDE's console window. The window has a dark background and a light blue title bar. The console output shows a red error message: "terminate called after throwing an instance of 'std::invalid_argument'" followed by a stack trace line: "what(): stoi".

```
terminate called after throwing an instance of 'std::invalid_argument'  
what(): stoi
```

2 - Analisando documentações do C++

- https://cplusplus.com/reference/?utm_source=chatgpt.com
- https://en.cppreference.com/?utm_source=chatgpt.com

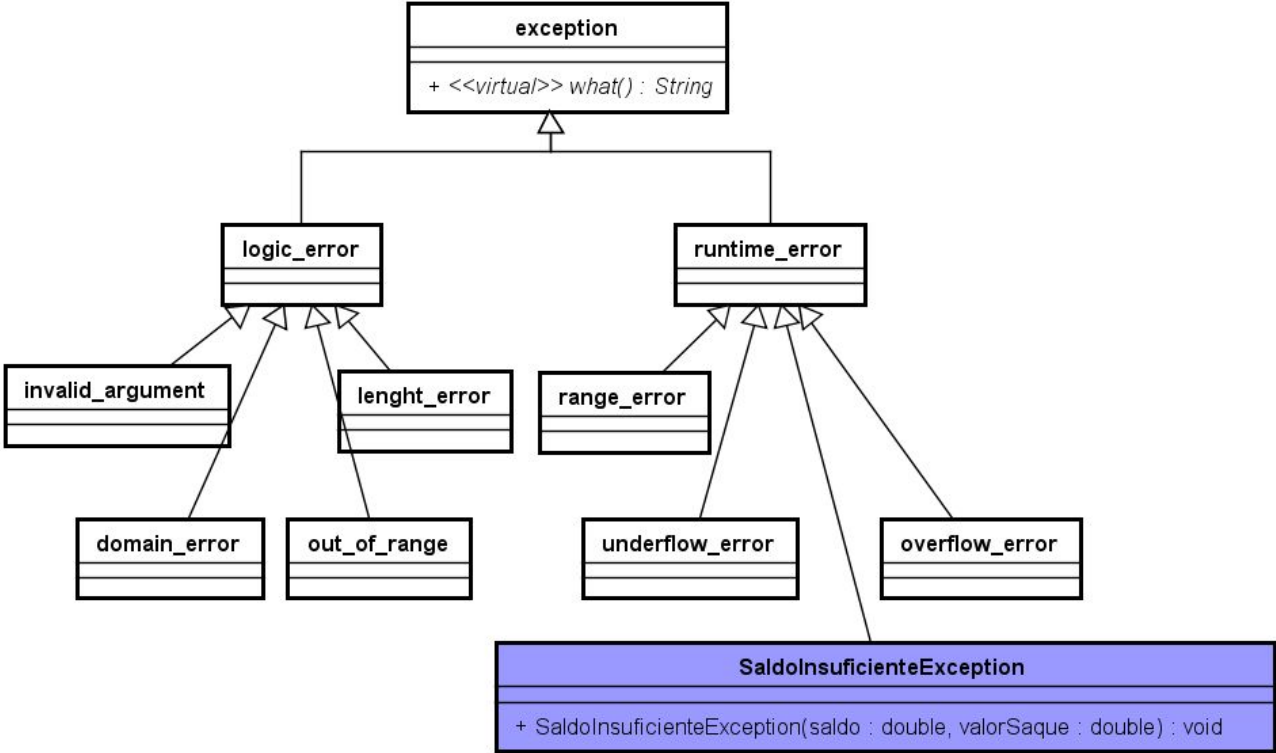


Posso criar minha própria classe ou até
hierarquia de exceções personalizada ?



Vamos imaginar que você está desenvolvendo um sistema bancário, e quer lançar uma exceção específica para **tentativas de saque com saldo insuficiente**.

Passo 1 - Criar a sua classe de Exceção



Passo 1: Criação da sua classe de exceção que herda da classe `runtime_error` do C++.

```
#include <stdexcept>
#include <string>

class SaldoInsuficienteException : public std::runtime_error {

public:

    SaldoInsuficienteException(double saldo, double valorSaque)

        : std::runtime_error("Saldo insuficiente: saldo atual = " +
std::to_string(saldo) + ", tentativa de saque = " +
std::to_string(valorSaque))
    {}

};
```

```
#include <iostream>
using namespace std;
```

```
class ContaBancaria {
private:
    double saldo;
```

```
public:
    ContaBancaria(double saldoInicial) : saldo(saldoInicial) {}

    void sacar(double valor) {
        if (valor > saldo) {
            throw SaldoInsuficienteException(saldo, valor);
        }
        saldo -= valor;
    }

    double getSaldo() const {
        return saldo;
    }
};
```

Passo 2 - Usar a classe criada para lançar exceções

Passo 3 - Tratar a exceção ao usar a função

```
int main() {
    ContaBancaria conta(100.0);

    try {

        conta.sacar(150.0); // Vai lançar exceção

    } catch (const SaldoInsuficienteException& e) {
        cerr << "Erro ao sacar: " << e.what() << endl;
    }

    return 0;
}
```

É uma boa prática manter as classes de exceção em arquivos separados

SaldoInsuficienteException.cpp

```
#ifndef SALDOINSUFICIENTEEXCEPTION_HPP
#define SALDOINSUFICIENTEEXCEPTION_HPP

#include <stdexcept>
#include <string>

class SaldoInsuficienteException : public std::runtime_error {
public:
    SaldoInsuficienteException(double saldo, double
    valorSaque)
        : std::runtime_error("Saldo insuficiente: saldo atual = "
        + std::to_string(saldo) + ", tentativa de saque = " +
        std::to_string(valorSaque)) {}
};

#endif
```

ContaBancaria.cpp

```
#ifndef CONTABANCARIA_HPP
#define CONTABANCARIA_HPP

#include "SaldoInsuficienteException.hpp"

class ContaBancaria {
private:
    double saldo;

public:
    ContaBancaria(double saldoInicial) : saldo(saldoInicial) {}

    void sacar(double valor) {
        if (valor > saldo) {
            throw SaldoInsuficienteException(saldo, valor);
        }
        saldo -= valor;
    }

    double getSaldo() const {
        return saldo;
    }
};

#endif
```

main.cpp

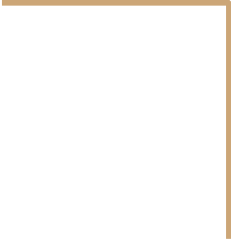
```
#include <iostream>
#include "ContaBancaria.hpp"

using namespace std;

int main() {
    ContaBancaria conta(100.0);
    try {

        conta.sacar(150.0);

    } catch (const SaldoInsuficienteException& e) {
        cerr << "Erro ao sacar: " << e.what() << endl;
    }
    return 0;
}
```



Também é possível colocar **atributos** em classes de exceção e **métodos** particulares de forma que você possa tratar essas exceções de forma mais personalizada.



```
#ifndef SALDOINSUFICIENTEEXCEPTION_HPP
#define SALDOINSUFICIENTEEXCEPTION_HPP

#include <stdexcept>
#include <string>

class SaldoInsuficienteException : public std::runtime_error {
private:

    double saldoAtual;
    double valorTentado;

public:
    SaldoInsuficienteException( double saldo, double valor)
        : std::runtime_error("Saldo insuficiente"),
          saldoAtual(saldo), valorTentado(valor) {}

    double getSaldoAtual() const { return saldoAtual; }

    double getValorTentado() const { return valorTentado; }

};

#endif
```

```
#ifndef CONTABANCARIA_HPP
#define CONTABANCARIA_HPP

#include "SaldoInsuficienteException.hpp"

class ContaBancaria {
private:
    double saldo;

public:
    ContaBancaria(double saldoInicial) : saldo(saldoInicial) {}

    void sacar(double valor) {
        if (valor > saldo) {
            throw SaldoInsuficienteException(saldo, valor);
        }
        saldo -= valor;
    }

    double getSaldo() const {
        return saldo;
    }
};

#endif
```

Aqui normal, nenhuma mudança

```
#include <iostream>
#include "ContaBancaria.hpp"

using namespace std;

int main() {

    ContaBancaria conta(100.0);

    try {
        conta.sacar(150.0);

    } catch (const SaldoInsuficienteException& e) {
        cerr << "Erro: " << e.what() << endl;
        cerr << "Saldo atual: " << e.getSaldoAtual() << endl;
        cerr << "Tentativa de saque: " << e.getValorTentado() << endl;

        if (e.getValorTentado() <= e.getSaldoAtual() + 50.0) {
            cerr << "Aviso: você pode ativar o limite especial." << endl;
        }
    }

    return 0;
}
```

A seguir um possível exemplo no contexto de biblioteca.
É apenas um “possível” exemplo, não significa que deve ser feito dessa forma no sistema de vocês.

Suponha que deseja-se controlar o nro de livros que podem ser emprestados por um determinado usuário. Se houver uma tentativa de empréstimo maior do que o permitido, isso deve ser adequadamente tratado por uma exceção específica

```
#include <iostream>
#include "Usuario.h"
#include "LimiteEmprestimosExcedido.h"
```

```
using namespace std;
```

```
int main() {
```

```
    Usuario usuario("Joao", 3);
```

```
    try {
```

```
        usuario.realizarEmprestimo(5);
```

```
    }
```

```
    catch (const LimiteEmprestimosExcedido& e) {
```

```
        cout << "ERRO: "
```

```
            << e.what()
```

```
            << endl;
```

```
        cout << "Usuario: "
```

```
            << e.getNomeUsuario()
```

```
            << endl;
```

```
        cout << "Limite permitido: "
```

```
            << e.getLimitePermitido()
```

```
            << endl;
```

```
        //continua ao lado
```

```
        cout << "Quantidade solicitada: "
              << e.getQuantidadeSolicitada()
              << endl;
```

```
        int excedente = e.getQuantidadeSolicitada() -
                        e.getLimitePermitido();
```

```
        cout << "Excedeu o limite em "
```

```
            << excedente
```

```
            << " livro(s)."
```

```
            << endl;
```

```
    }
```

```
    cout << endl;
```

```
    cout << "Programa continua normalmente."
```

```
        << endl;
```

```
    return 0;
```

```
}
```

```
#ifndef USUARIO_H
#define USUARIO_H

#include <string>

using namespace std;

class Usuario {
private:
    string nome;
    int limiteEmprestimos;

public:

    Usuario(string nome, int limite);

    void realizarEmprestimo(int quantidadeLivros);
};

#endif
```

Usuario.h

```
#include "Usuario.h"
#include "LimiteEmprestimosExcedido.h"
#include <iostream>

using namespace std;

Usuario::Usuario(string nome, int limite) {
    this->nome = nome;
    this->limiteEmprestimos = limite;
}

void Usuario::realizarEmprestimo(int quantidadeLivros) {

    if (quantidadeLivros > limiteEmprestimos) {

        throw LimiteEmprestimosExcedido(nome, limiteEmprestimos,
quantidadeLivros);
    }

    cout << "Emprestimo realizado com sucesso!"
        << endl;
}
```

Usuario.cpp

```
#ifndef LIMITEEMPRESTIMOSEXCEDIDO_H
#define LIMITEEMPRESTIMOSEXCEDIDO_H
#include <string>
#include <exception>
using namespace std;

class LimiteEmprestimosExcedido : public exception {
private:
    string nomeUsuario;
    int limitePermitido;
    int quantidadeSolicitada;

public:
    LimiteEmprestimosExcedido(string nome, int limite, int quantidade) {
        nomeUsuario = nome;
        limitePermitido = limite;
        quantidadeSolicitada = quantidade;
    }

    const char* what() const noexcept override {
        return "Limite de empréstimos excedido";
    }

    string getNomeUsuario() const {
        return nomeUsuario;
    }

    int getLimitePermitido() const {
        return limitePermitido;
    }

    int getQuantidadeSolicitada() const {
        return quantidadeSolicitada;
    }
};
```

Na classe exception:

virtual const char what() const noexcept;*

const char* - o retorno é um ponteiro constante
const - nenhum atributo é modificado dentro
noexcept - garante que este método não pode lançar novas exceções.. se o próprio `what` lançasse exceções o tratamento ficaria caótico

LimiteEmprestimoExcedido.cpp



Você também pode usar mais de um Catch
para capturar exceções específicas



```

#include <iostream>
#include <stdexcept>
#include <string>

using namespace std;

// Classe de exceção personalizada para saldo insuficiente
class SaldoInsuficiente : public runtime_error {

    double saldoAtual;
    double valorTentado;

public:

    SaldoInsuficiente(double saldo, double valor)
        : runtime_error("Saldo insuficiente para o saque."),
          saldoAtual(saldo), valorTentado(valor) {}

    double getSaldoAtual() const { return saldoAtual; }
    double getValorTentado() const { return valorTentado; }
};

```

```

// Classe ContaBancaria
class ContaBancaria {
    string titular;
    double saldo;

public:
    ContaBancaria(const string& nome, double
saldoInicial)
        : titular(nome), saldo(saldoInicial) {}

    void sacar(double valor) {
        if (valor <= 0) {
            throw invalid_argument("Valor de saque deve
ser positivo.");
        }
        if (valor > saldo) {
            throw SaldoInsuficiente(saldo, valor);
        }
        saldo -= valor;
    }

    void depositar(double valor) {
        if (valor <= 0) {
            throw invalid_argument("Valor de depósito
deve ser positivo.");
        }
        saldo += valor;
    }

    double consultarSaldo() const {
        return saldo;
    }
}

```

Método sacar() lança dois tipos de exceção

```
int main() {
    ContaBancaria conta("João", 1000);

    try {
        cout << "Tentando sacar 1500..." << endl;
        conta.sacar(1500); // Deve lançar SaldoInsuficiente

    } catch (const SaldoInsuficiente& e) {
        cout << "Exceção: " << e.what() << endl;
        cout << "Saldo atual: " << e.getSaldoAtual() << ", valor tentado: " << e.getValorTentado() <<
endl;

    } catch (const invalid_argument& e) {
        cout << "Argumento inválido: " << e.what() << endl;

    } catch (const exception& e) {
        // Captura outras exceções derivadas de std::exception
        cout << "Erro genérico: " << e.what() << endl;

    } catch (...) {
        // Captura qualquer outra exceção
        cout << "Exceção desconhecida!" << endl;
    }

    cout << "Programa continua normalmente." << endl;
    return 0;
}
```

Código cliente preparado para tratar vários tipos de exceção que podem ser lançados pelo mesmo método sacar()

O catch(exception e) captura apenas exceções que herdam de exception.

O catch (...) funciona realmente ... significa, pegue qualquer outra exceção que não tenha sido capturada anteriormente.

Exercícios

Exercício 1

- Escreva um programa em C++ que leia um número inteiro positivo do usuário. Se o número informado for negativo, lance uma exceção do tipo `std::invalid_argument`.
- Capture a exceção e mostre a mensagem de erro.
- Pode ser tudo no mesmo arquivo.

Exercício 2

- Implemente um código em C++ que crie um `std::vector<int>` vazio e tente acessar o elemento na posição 0 usando `at(0)`. Capture a exceção lançada automaticamente pelo C++ e imprima a mensagem de erro.

Exercício 3

- Crie uma classe chamada `TemperaturaInvalidaException`, derivada de `std::runtime_error`, que contenha um atributo `valorInformado`.
- Depois, crie uma função `verificarTemperatura(double)` que lance essa exceção caso a temperatura esteja fora do intervalo aceitável (por exemplo, entre -50 e 100 graus Celsius).
- No `main`, chame a função e trate a exceção imprimindo a mensagem e o valor informado.